TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO
PBL4 – DỰ ÁN HỆ THỐNG THÔNG MINH

TÊN ĐỀ TÀI:
HỆ THỐNG MÁY DIỆT CỎ BẰNG LASER

Giảng viên hướng dẫn: **TS. Ninh Khánh Duy**

HỌ VÀ TÊN SINH VIÊN	LỚP HỌC PHẦN
Phan Đình Hồi	23.16A
Nguyễn Văn Lợi	23.16A
Nguyễn Thanh Hậu	23.16A

ĐÀ NẴNG, 12/2025

TÓM TẮT ĐỒ ÁN

Đồ án tập trung giải quyết vấn đề cấp thiết về ô nhiễm môi trường do lạm dụng thuốc diệt cỏ và sự thiếu hụt nhân lực trong nông nghiệp bằng cách phát triển hệ thống Robot diệt cỏ tự hành thông minh hướng tới nông nghiệp chính xác. Nhóm thực hiện đề xuất giải pháp tích hợp trí tuệ nhân tạo và thị giác máy tính trên thiết bị nhúng biên. Phương pháp tiếp cận cốt lõi sử dụng mô hình học sâu YOLOv8 trên vi xử lý Jetson Nano để phát hiện cỏ dại thời gian thực, kết hợp kỹ thuật Camera Stereo để tìm độ sâu nhằm xác định vị trí cỏ thật từ camera. Về mặt quản lý, hệ thống vận hành trên kiến trúc Web Server sử dụng Java Servlet được triển khai trên máy chủ Apache Tomcat, áp dụng giao thức giao tiếp lai giữa Socket TCP (điều khiển độ trễ thấp) và HTTP API (truyền tải dữ liệu lớn). Kết quả thực nghiệm cho thấy Robot di chuyển ổn định, nhận diện và xử lý cỏ dại hiệu quả, đồng thời cung cấp giao diện giám sát trực quan, cho phép truy vết lịch sử và thống kê hiệu suất hoạt động cho người dùng.

BẢNG PHÂN CÔNG NHIỆM VỤ

Sinh viên thực hiện	**Các nhiệm vụ**	**Tự đánh giá theo 3 mức (Đã hoàn thành/Chưa hoàn thành/ Không triển khai)**
Phan Đình Hồi	- Xây dựng giao diện web hiển thị kết quả cho người dùng. - Thiết kế HTTP RESTful API để giao tiếp giữa web và hệ thống xử lý. - Thiết kế cơ sở dữ liệu để lưu thông tin người dùng và phiên hoạt động của máy.	Đã hoàn thành
Nguyễn Văn Lợi	- Thiết kế và lắp đặt hệ thống phần cứng IoT. - Kết nối camera/cảm biến thu thập dữ liệu ảnh. - Gửi dữ liệu từ thiết bị IoT về server xử lý. - Đồng bộ hoạt động với hệ thống AI và web.	Đã hoàn thành
Nguyễn Thanh Hậu	- Gán nhãn dữ liệu cho bài toán segmentation. - Thu thập dữ liệu ảnh cỏ. - Huấn luyện mô hình Yolov8-segmentation. - Đánh giá model. - Triển khai xử lí trên jetson (model deployment).	Đã hoàn thành

MỤC LỤC

TÓM TẮT ĐỒ ÁN **2**
BẢNG PHÂN CÔNG NHIỆM VỤ **3**
MỤC LỤC **5**
MỤC LỤC HÌNH ẢNH **7**
1. GIỚI THIỆU **9**
1.1. Tổng quan **9**
1.2. Mục đích **9**
1.3. Phương pháp **10**
2. GIẢI PHÁP **11**
2.1. Tổng quan hệ thống **11**
2.2. Nguyên lý hoạt động của hệ thống **12**
2.3. Giải pháp về phần cứng **14**
2.3.1. Thực hiện chiếu laser và di chuyển xe 14
2.3.2. Xử lý thông tin 16
2.3.3. Thu thập hình ảnh 17
2.4. Giải pháp về truyền thông **18**
2.4.1. Giao tiếp nội bộ (Kết nối có dây) 18
2.4.2. Giao tiếp hệ thống (Kết nối không dây) 18
2.5. Giải pháp về phần mềm **19**
2.5.1. Hệ thống camera stereo. 19
2.6. Giải pháp nhận diện cỏ **25**
2.6.1. Lựa chọn mô hình 25
2.6.2. Tổng quan về mô hình sử dụng 27
2.6.3. Xử lý nhận diện 28
2.6.4. Huấn luyện và đánh giá 28
3. KẾT QUẢ **34**
3.1. Các điều kiện tiến hành thực nghiệm **34**
3.2. Thông số kỹ thuật của Jetson Nano khi chạy model **35**
3.3. Kết quả web hệ thống máy diệt cỏ **36**
3.4. Đánh giá **37**
3.4.1. Hệ thống IoT 37

3.4.2. Đánh giá về mô hình AI 40
3.4.3. Đánh giá về Web Server 41
3.5. Bảng thống kê chi phí 42
4. HƯỚNG PHÁT TRIỂN 43
4.1. IoT 43
4.2. Web Server 43
4.3. Model AI 44
TÀI LIỆU THAM KHẢO 45

MỤC LỤC HÌNH ẢNH

Hình 1. Sơ đồ hệ thống 12
Hình 2. Sơ đồ khối mô-đun phần cứng 14
Hình 3. Sơ đồ khối phần cứng 15
Hình 4. Cảm biến dò line bố trí trên xe 16
Hình 5. Thông tin về phần cứng jetson Nano 16
Hình 6. Camera bố trí trên xe 17
Hình 7. Mô phỏng camera stereo 21
Hình 8. Hiệu chỉnh camera để các điểm trên ảnh trái trùng vs chính nó trên ảnh phải 21
Hình 9. Thuật toán local block matching 22
Hình 10. Sử dụng bàn cờ hiệu chỉnh camera 24
Hình 11. Kết quả tạo độ chênh lệch giữa các điểm ảnh 24
Hình 12. Rau bạc hà 25
Hình 13. Kết quả nhận diện cỏ 26
Hình 14. Tọa độ điểm chiếu laser 26
Hình 15. Sơ đồ luồng xử lý 28
Hình 16. Lô dữ liệu đầu vào 29
Hình 17. Kết quả tổng thể huấn luyện 29
Hình 18. Precision-recall curve 30
Hình 19. F1-confidence curve 31
Hình 20. F1-confidence curve 31
Hình 21. Chia điểm trên khung xương 32
Hình 22. Kết quả xử lý 33
Hình 23. Kết quả theo tỉ lệ che phủ 33
Hình 24. Kết quả theo số điểm bắn 34
Hình 25. Thông số kĩ thuật của jetson khi chạy với model 35
Hình 26. Độ chính xác trung bình khi chạy test 36
Hình 27. Entity-relationship diagram – ERD 36
Hình 28. Giao diện trang danh sách máy 36
Hình 29. Giao diện chi tiết máy khi đã kết nối 37
Hình 30. Giao diện lịch sử cỏ của phiên 37
Hình 31. Đánh giá độ chính xác laser 39
Hình 32. Đánh giá mô hình AI 40
Hình 33. Confusion matrix khi test 41

1. GIỚI THIỆU

1.1. Tổng quan

Trong đề tài này, nhóm chúng em sẽ xây dựng hệ thống Robot diệt cỏ tự hành sử dụng năng lượng Laser để thay thế việc sử dụng thuốc trừ sâu hóa học trong nông nghiệp.

Hệ thống được xây dựng dựa trên nền tảng công nghệ thị giác máy tính (Computer Vision) và Trí tuệ nhân tạo (AI), cụ thể sử dụng mô hình học sâu YOLOv8 trên thiết bị nhúng Jetson Nano để nhận diện cỏ, kết hợp với công nghệ camera stereo tìm chiều sâu giúp xác định vị trí của cỏ và hệ thống Web Server để giám sát từ xa.

1.2. Mục đích

- Về Robot & AI: Xây dựng mô hình xe tự hành có khả năng di chuyển ổn định theo luống, ứng dụng mô hình học sâu YOLOv8 để phát hiện chính xác cỏ dại trong thời gian thực trên thiết bị nhúng biên.
- Về Công nghệ Xử lý ảnh 3: Nghiên cứu và ứng dụng hệ thống Camera Stereo (Stereo Vision) để xây dựng bản đồ chiều sâu (Depth Map), thực hiện chuyển đổi tọa độ ảnh 2D sang tọa độ không gian 3D (x, y, z) để dẫn hướng chính xác cho tia Laser.
- Về Hệ thống Web: Xây dựng hệ thống giám sát và điều khiển trung tâm thông qua Web App, sử dụng công nghệ Java Servlet và JSP được triển khai trên máy chủ ứng dụng Apache Tomcat.
- Về Giao thức truyền thông: Nghiên cứu và ứng dụng cơ chế giao tiếp lai (Hybrid Communication) để tối ưu hiệu năng hệ thống:
    - Sử dụng Socket TCP/IP cho kênh điều khiển thời gian thực (độ trễ thấp) để gửi lệnh Start/Stop và cập nhật trạng thái Robot.
    - Sử dụng HTTP RESTful API cho kênh truyền tải dữ liệu lớn để upload hình ảnh bằng chứng và đồng bộ lịch sử diệt cỏ lên Server.
- Về Dữ liệu: Tìm hiểu cách tổ chức cơ sở dữ liệu MySQL để lưu trữ phiên làm việc, truy vết lịch sử hoạt động và thống kê hiệu quả diệt cỏ.

1.3. Phương pháp

a) Đối tượng

- Các phương pháp, thuật toán thị giác máy tính: Tập trung vào bài toán phát hiện vật thể (Object Detection) và phân đoạn ảnh để phân biệt cỏ dại và cây trồng.
- Mô hình học sâu (Deep Learning): Nghiên cứu cấu trúc và cách huấn luyện mô hình YOLOv8 để tối ưu hóa tốc độ nhận diện trên thiết bị nhúng biên.
- Công nghệ Stereo Vision: Nghiên cứu nguyên lý hoạt động của Camera kép (Stereo Camera) và ma trận tính toán bản đồ chiều sâu (Depth Map) để xác định tọa độ không gian 3D.
- Công nghệ Web và IoT: Nghiên cứu kiến trúc Java Servlet trên nền tảng Apache Tomcat, cùng các giao thức truyền thông mạng (Socket TCP, HTTP API) để xây dựng hệ thống điều khiển từ xa.

b) Phạm vi nghiên cứu

Tập trung vào bài toán nhận diện cỏ dại (Weed Detection): Nghiên cứu, huấn luyện và tối ưu hóa mô hình học sâu YOLOv8 để phân biệt đối tượng "Cỏ dại" và "Cây trồng" trong điều kiện môi trường nông nghiệp giả lập (ánh sáng tự nhiên, nền đất).

Định vị và xử lý không gian 3D: Giới hạn nghiên cứu trong việc sử dụng Camera Stereo để tính toán bản đồ chiều sâu (Depth Map) và chuyển đổi tọa độ điểm ảnh sang tọa độ thực tế (x, y, z) trong phạm vi gần (< 1 mét).

Công nghệ điều khiển và giao tiếp: Tập trung xây dựng cơ chế giao tiếp lai (Hybrid Communication) trong mạng nội bộ (LAN/Wifi): sử dụng Socket TCP cho các lệnh điều khiển thời gian thực (Start/Stop) và HTTP API cho việc truyền tải dữ liệu hình ảnh.

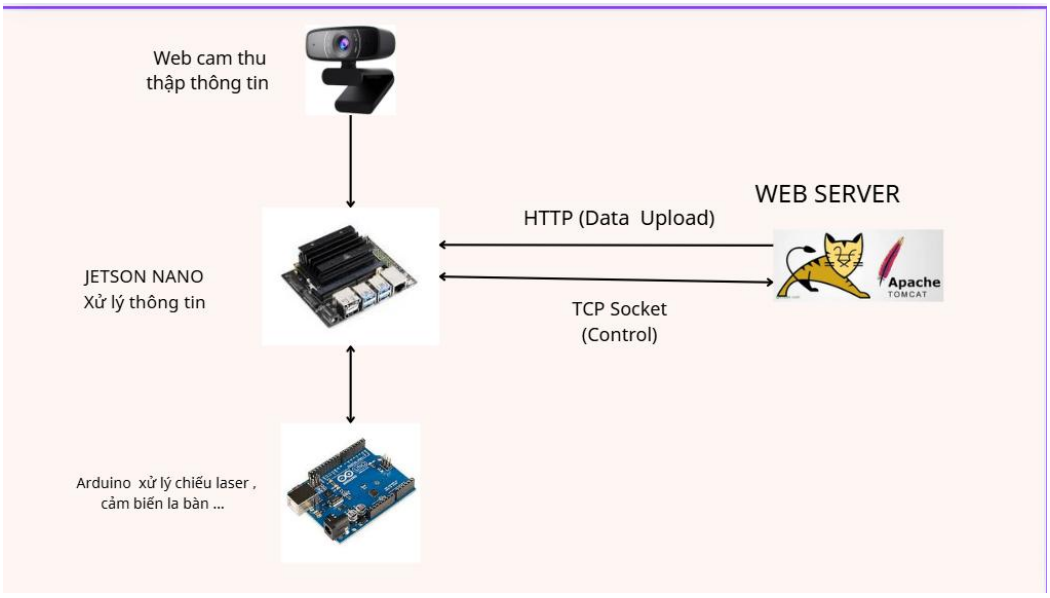
Hệ thống giám sát: Xây dựng ứng dụng Web trên nền tảng Java Servlet chạy trên máy chủ Apache Tomcat, tập trung vào các chức năng cốt lõi: xem luồng video trực tiếp (Streaming), lưu trữ bằng chứng diệt cỏ và thống kê hiệu suất hoạt động.

2. GIẢI PHÁP

2.1. Tổng quan hệ thống

- Khối Thu thập và Xử lý trung tâm (Jetson Nano & Camera):
  - Đầu vào: Module Camera (Stereo Vision) thu thập hình ảnh môi trường thời gian thực và gửi tín hiệu về bộ xử lý trung tâm.
  - Xử lý: Máy tính nhúng Jetson Nano đóng vai trò là "bộ não" của Robot. Tại đây, mô hình YOLOv8 sẽ phân tích hình ảnh để nhận diện cỏ, đồng thời thuật toán xử lý ảnh 3D sẽ tính toán tọa độ không gian của mục tiêu.
- Khối Điều khiển cấp thấp (Arduino):
  - Nhận tọa độ và lệnh điều khiển từ Jetson Nano thông qua giao tiếp Serial (UART).
  - Trực tiếp điều khiển các cơ cấu chấp hành: Động cơ di chuyển, Servo Pan-Tilt để hướng tia Laser, và đóng ngắt Laser.
- Khối Quản lý từ xa (Web Server - Apache Tomcat):
  - Hệ thống máy chủ chạy trên nền tảng Apache Tomcat, giao tiếp với Robot qua hai kênh riêng biệt để tối ưu hiệu năng:
    - Kênh TCP Socket (Control): Duy trì kết nối liên tục để gửi các lệnh điều khiển tức thời (Start, Stop, Pause) và nhận cập nhật trạng thái xe với độ trễ thấp nhất.

- Kênh HTTP (Data Upload): Sử dụng API để nhận các gói dữ liệu lớn như hình ảnh bằng chứng (Evidence Images) và nhật ký hoạt động để lưu trữ vào cơ sở dữ liệu, tránh gây tắc nghẽn cho luồng điều khiển.



Hình 1. Sơ đồ hệ thống

2.2. Nguyên lý hoạt động của hệ thống

Hệ thống hoạt động dựa trên sự phối hợp chặt chẽ giữa ba khối chức năng: Khối Nhúng biên (Robot), Khối Máy chủ (Server) và Khối Người dùng (Web Client). Nguyên lý hoạt động tổng thể được chia thành các giai đoạn sau:

- **Giai đoạn 1: Khởi tạo và Kết nối (Initialization)**

- Ngay khi được cấp nguồn, vi xử lý trung tâm Jetson Nano sẽ khởi động hệ điều hành và tự động chạy chương trình Client (Python). Robot thực hiện quy trình bắt tay (Handshake) với Máy chủ Apache Tomcat thông qua giao thức Socket TCP/IP. Sau khi xác thực mã định danh (Device ID), Robot chuyển sang trạng thái "Chờ lệnh" (Idle) và liên tục gửi gói tin Heartbeat để duy trì kết nối.

- **Giai đoạn 2: Di chuyển và Định hướng (Navigation & Stabilization)**

- Khi nhận lệnh START từ Web App, Robot bắt đầu di chuyển. Hệ thống sử dụng thuật toán hợp nhất cảm biến (Sensor Fusion) để duy trì quỹ đạo:

  ✓ Cảm biến La bàn số liên tục đo góc lệch hướng (Heading) của xe so với hướng chuẩn.

  ✓ Bộ điều khiển PID sẽ tính toán sai số góc lệch và điều chỉnh tốc độ chênh lệch giữa hai bánh xe động cơ DC, giúp xe di chuyển thẳng hàng ngay cả khi địa hình gồ ghề hoặc vạch luống bị đứt quãng.

- **Giai đoạn 3: Nhận diện và Định vị mục tiêu (Perception & Localization)**

Trong quá trình di chuyển, hệ thống Camera Stereo liên tục thu thập hình ảnh môi trường và đưa vào luồng xử lý song song:

- Phát hiện (Detection): Ảnh từ camera trái được đưa qua mô hình học sâu YOLOv8. Nếu phát hiện đối tượng thuộc lớp "Weed" (Cỏ dại), mô hình trả về khung bao (Bounding Box) và vùng phân đoạn được xác định là đối tượng trong ảnh (Segment)
- Tinh chỉnh (Refinement): Trong vùng phân đoạn (vùng pixel), thuật toán K-means Clustering phân tích mật độ điểm ảnh để loại bỏ nhiễu nền đất, xác định chính xác trọng tâm (Centroid) của cây cỏ.
- Định vị 3D (3D Mapping): Hệ thống tính toán bản đồ chênh lệch (Disparity Map) giữa hai ảnh trái/phải tại điểm trọng tâm, từ đó suy ra khoảng cách chiều sâu (Z). Kết hợp với ma trận nội tham camera, tọa độ ảnh 2D (u, v) được chuyển đổi sang tọa độ không gian thực tế 3D (x, y, z) so với gốc Robot.

- **Giai đoạn 4: Tác động xử lý (Actuation)**

Dựa trên tọa độ (x, y, z) đã tính toán, hệ thống áp dụng bài toán Động học nghịch (Inverse Kinematics) để tính ra góc quay cần thiết cho 2 động cơ Servo (trục Pan và Tilt). Đầu phóng Laser được điều hướng chính xác đến mục tiêu và kích hoạt chùm tia công suất cao trong thời gian định trước để tiêu diệt cỏ.

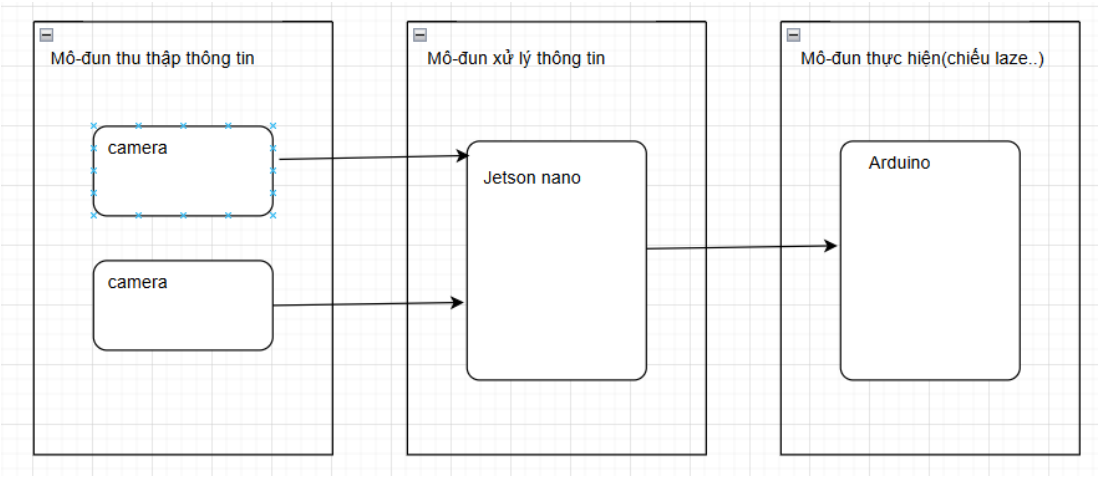
- **Giai đoạn 5: Báo cáo và Đồng bộ dữ liệu (Reporting)**

  Sau khi xử lý xong, Robot thực hiện quy trình báo cáo kép:

- Gửi thông báo tức thời: Gửi gói tin WEED qua kênh Socket để Web cập nhật số lượng cỏ trên biểu đồ thời gian thực.
- Gửi bằng chứng: Gửi hình ảnh hiện trường qua giao thức HTTP POST (Multipart) lên RESTful API của Server để lưu trữ vào cơ sở dữ liệu, phục vụ việc truy xuất lịch sử sau này.

2.3. Giải pháp về phần cứng

Chia phần cứng thành 3 mô-đun chính, mô-đun thực hiện chiếu laser và di chuyển xe, mô-đun xử lý thông tin và mô-đun nhận thông tin.

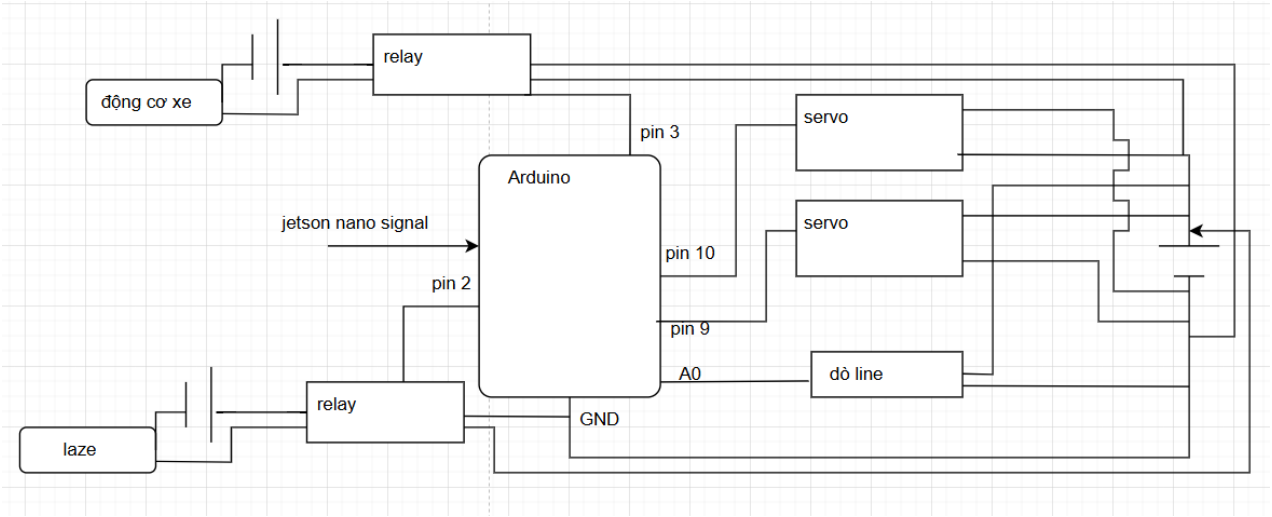


Hình 2. Sơ đồ khối mô-đun phần cứng

2.3.1. Thực hiện chiếu laser và di chuyển xe

Arduino nhận tin nhắn từ Jetson Nano thông qua giao tiếp UART, có thể sử dụng các chân GPIO của Jetson Nano để thay thế Arduino cũng được nhưng nếu tách biệt thành hai bộ phận như thế này thì sẽ dễ dàng sửa chửa và mở rộng, ngoài ra Arduino có các chân

đọc tín hiệu analog và chân PWM mà trên Jetson Nano không có.



Hình 3. Sơ đồ khối phần cứng

a) Bộ phận chiếu laser

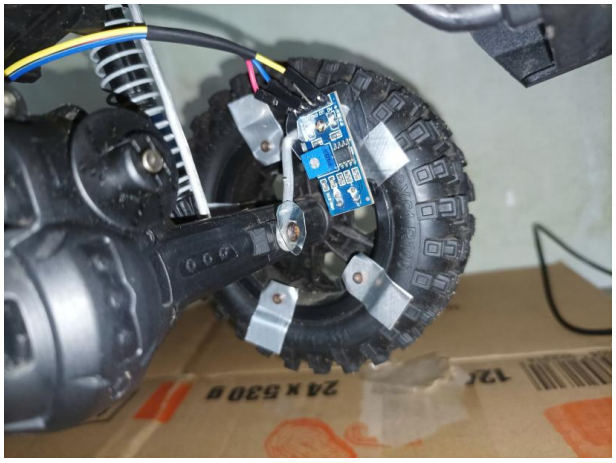
- Sử dụng bộ pan – tilt servo để chiếu các hướng laser tới các vị trí mặt đất cấu tạo của pan – tilt :
- Servo Pan (Ngang): Lắp ở trục đứng (trục Z), chịu trách nhiệm quay laser sang trái hoặc phải (Pan). Nó cung cấp chuyển động theo phương ngang.
- Servo Tilt (Nghiêng): Lắp trên giá đỡ của servo Pan, chịu trách nhiệm quay laser lên hoặc xuống (Tilt). Nó cung cấp chuyển động theo phương dọc.
- Để chiếu laser đến vị trí cỏ ta chỉ gần xác định góc quay của hai servo, giá trị hai góc quay này sẽ được tính toán từ bộ phận tính toán (Jetson nano). Arduino sẽ nhận hai giá trị góc này và điều khiển hai servo đến đúng vị trí.

b) Bộ phận di chuyển xe.

Xe di chuyển một đoạn sao cho đi tiếp vùng cỏ xử lý tiếp theo, khoảng cách này sẽ được tính toán dựa trên độ rộng pixel ảnh mà ta chụp xử lý cùng với chiều sao giữa camera và mặt đất, để xe di chuyển với một khoảng xác định cần bộ đo chiều dài xe đi được, nó tương tự như kim đo km trên xe máy, giải pháp là sử dụng cảm biến dò line và đếm các số vòng dán trên bánh.

- Kích thước của bánh xe là 34:cm

- Số vạch đếm: 5

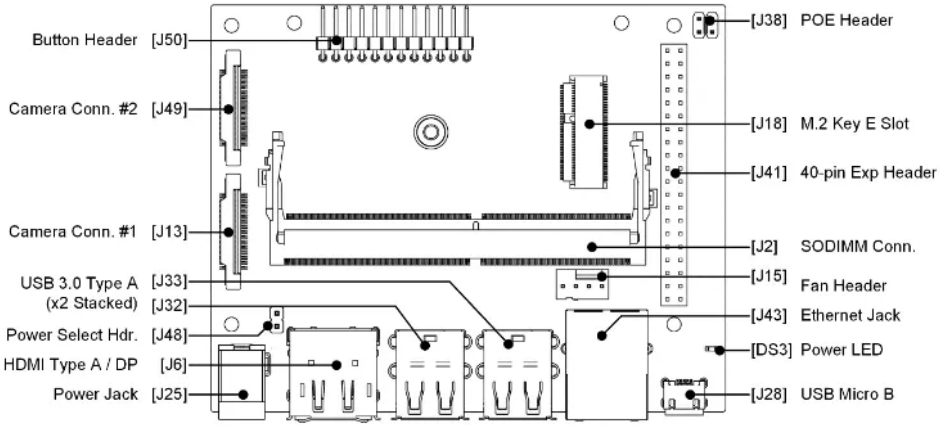


Hình 4. Cảm biến dò line bố trí trên xe

Sử dụng Module Cảm Biến Dò Line Đơn TCRT5000 4 Chân để đếm các vạch.

2.3.2. Xử lý thông tin

Mục đích chạy model Al, thu thập ảnh tử camera, xử lý tìm độ sâu đối với mỗi điểm trên ảnh sử dụng thuật toán stereo camera, gửi kết quả về server… để xử lý realtime cần một bộ kit phát triển mạnh mẽ nên sử dụng Jetson Nano là một giải pháp khá tối ưu.



Hình 5. Thông tin về phần cứng Jetson Nano

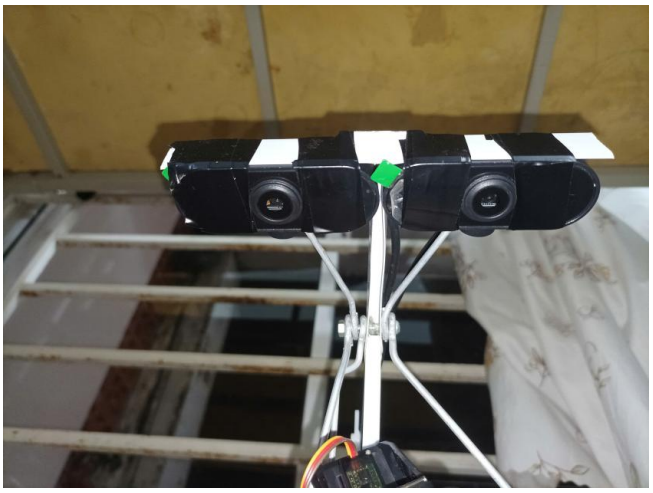
Thông số kĩ thuật của Jetson nano:

Danh mục	Thông số kỹ thuật
Hiệu suất AI	472 GFLOPS

GPU (Bộ xử lý đồ họa)	128 nhân CUDA Kiến trúc Maxwell
CPU (Bộ xử lý trung tâm)	Quad-core ARM Cortex-A57
Bộ nhớ (RAM)	4 GB 64-bit LPDDR4
Bộ nhớ trong (Lưu trữ)	Khe cắm MicroSD
Mã hóa Video	4K @ 30 (H.264/H.265)
Giải mã Video	4K @ 60 (H.264/H.265)

2.3.3. Thu thập hình ảnh.

Sử dụng hai webcam XWF-1080P một cái để thu thập ảnh cho model yolo-v8 nhận diện một cái còn lại sẽ kết hợp tạo thành hệ thống camera stereo giúp xác định độ chiều cao từ camera đến vị trí của cỏ nhằm xác định tọa độ để bắn laser.



Hình 6. Camera bố trí trên xe

Hai camera được lặp cắm vào hai cổng usb của jetson nano, vì kinh phí nên nhóm sử dụng webcam nên có một độ trể nhất định, giải pháp tốt hơn là là sử dụng camera kết nối vào hai cổng camera conn của Jetson Nano để có độ chính xác và độ trể thấp hơn.

Thông số kỹ thuật của webcam XWF-1080P:

Thông số kỹ thuật	Giá trị

Độ phân giải	640x480
Tiêu chuẩn Video	VGA
Tỷ lệ khung hình (FPS)	17 FPS
Webcam MegaPixels	0.31 MP
Loại luồng (Stream Type)	video
Chế độ ảnh (Image Mode)	rgb
Tỷ lệ khung hình (Aspect Ratio)	1.33
Tốc độ bit	4.3 MB/s

2.4. Giải pháp về truyền thông

Hệ thống sử dụng mô hình truyền thông phân tán theo kiến trúc Client-Server, kết hợp giữa các kết nối vật lý nội bộ (Wired) để đảm bảo tốc độ xử lý phần cứng và kết nối mạng không dây (Wireless) để đảm bảo tính linh hoạt, giám sát từ xa.

2.4.1. Giao tiếp nội bộ (Kết nối có dây)

Đây là các kết nối vật lý giữa các module phần cứng bên trong Robot để đảm bảo tín hiệu điều khiển ổn định và băng thông lớn cho xử lý ảnh:

- Giao tiếp Camera - Máy tính nhúng: Camera Stereo kết nối với Jetson Nano thông qua cổng USB 3.0 hoặc chuẩn CSI (Camera Serial Interface). Đây là kết nối yêu cầu băng thông cao nhất để truyền tải luồng video kép (Dual Stream) phục vụ tính toán bản đồ chiều sâu (Depth Map).
- Giao tiếp Máy tính nhúng - Vi điều khiển: Jetson Nano gửi lệnh điều khiển động cơ và nhận dữ liệu cảm biến (Encoder, La bàn) từ Vi điều khiển (Arduino/STM32) thông qua giao thức UART (Serial Communication) qua cáp USB hoặc chân GPIO (Tx/Rx).

2.4.2. Giao tiếp hệ thống (Kết nối không dây)

Robot kết nối với Máy chủ (Web Server) thông qua mạng Wi-Fi (MẠNG LAN)Để tối ưu hóa hiệu năng giữa việc điều khiển thời gian thực và truyền tải dữ liệu lớn, nhóm đề xuất giải pháp Giao tiếp lai (Hybrid Communication):

- Kênh Điều khiển (Control Channel) - Sử dụng Socket TCP/IP:
  - Cơ chế: Thiết lập một kênh kết nối hai chiều liên tục (Persistent Connection) giữa Client (Robot) và Server (Java Application).
  - Mục đích: Truyền tải các gói tin lệnh kích thước nhỏ nhưng yêu cầu độ trễ cực thấp (Low Latency).
  - Ứng dụng: Gửi lệnh START, STOP, PAUSE từ Web xuống Robot và cập nhật trạng thái STATUS (Online/Offline/Running) từ Robot lên Web.
- Kênh Dữ liệu (Data Channel) - Sử dụng HTTP RESTful API:
  - Cơ chế: Sử dụng mô hình Request-Response thông qua giao thức HTTP (POST Method).
  - Mục đích: Truyền tải các gói dữ liệu kích thước lớn, không yêu cầu tính thời gian thực quá khắt khe nhưng cần độ tin cậy cao và khả năng xử lý file.
  - Ứng dụng: Robot upload hình ảnh bằng chứng (Evidence Images) và thông tin thống kê (vị trí cỏ, số lượng) lên Server thông qua API

2.5. Giải pháp về phần mềm

2.5.1. Hệ thống camera stereo.

a) Lý do tại sao cần phải tái tọa không gian 3D.

Thứ nhất, Cần có thông tin để xử lý cỏ nằm ở đâu. → giải pháp: sử dụng 1 camera để thu thập thông tin.

Thứ hai, khi biết cỏ nằm ví trí nào trong ảnh rồi cần điều khiển laser đến vị trí cỏ đó không biết điều khiển như thế nào. vì mặt phẳng camera chụp không phải lúc nào cũng bằng phản và khoản cách từ camera đến mặt phẳng có thể cao hoặc thấp nên không thể

ước tính chiều cao h cụ thể → giải pháp: kết hợp thêm 1 camera để tạo thành camera đôi nhằm mục đích sử dụng phương pháp đo khoảng cách bằng camera stereo từ đó giúp cho bộ xử lý biết 1 điểm bất kì nằm ở đâu trong không gian thật.

b) Camera stereo

Nếu từ không gian 3D ánh xạ lên hình ảnh ta có công thức như sau.

$$u = f_x \frac{x}{z} + o_x , v = f_y \frac{y}{z} + o_y$$

Với f là tiêu cự, x, y, z là vị trí của vật so với tọa độ của camera. o_x , o_y là độ lệnh giữa tâm màn ảnh với khẩu độ camera. ở đây f_x, f_y, o_x, o_y là một hằng số.

Nếu từ dữ liệu trên ta chuyển đổi sang không gian 3D thì ta được như sau:

$$x = (u - o_x)\frac{z}{f_x} , y = (v - o_y)\frac{z}{f_y}, z > 0$$

Ta thấy chưa thể tìm ra được Z là độ sâu cần tìm.

Với trường hợp sử dụng camera đôi.

Ta suy ra được công thức sau:

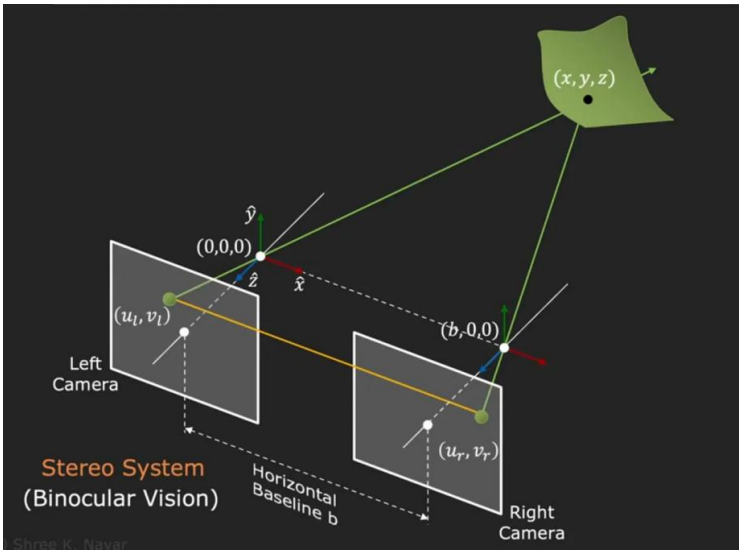
$u_l = f_x \frac{x}{z} + o_x , v_l = f_y \frac{y}{z} + o_y$

$u_r = f_x \frac{x-b}{z} + o_x, v_r = f_y \frac{y}{z} + o_y$

Sử dụng camera đôi thì ta sẽ suy ra hiệu giữa hai điểm trên hai ảnh của trục x, hay nói cách khác là suy ra độ lệnh của một điểm trên ảnh này với ảnh kia.

⇨ Ta có độ lệnh là $d = u_l - u_r$

Từ công thức trên ta thay vào : $d = \left(f_x \frac{x}{z} + o_x\right) - \left(f_x \frac{x-b}{z} + o_x\right) => d = f_x \frac{b}{z} \leftrightarrow z = \frac{f_x b}{d}$



Hình 7. Mô phỏng camera stereo

c) Cách tìm độ chênh lệnh của một điểm pixel.

- Làm sao để tìm một điểm trên ảnh trái nằm ở chổ nào bên ảnh phải.
- Để đơn giản việc tìm kiếm ta cần tiến hành một bước đó là hiệu chỉnh hình ảnh sao cho vị trí của mỗi điểm ta tìm trên ảnh trái có tạo độ dọc y_1 nó bằng với tạo độ dọc y_2 trên ảnh phải.

Hình 8. Hiệu chỉnh camera để các điểm trên ảnh trái trùng vs chính nó trên ảnh phải

Lúc này $y_l = y_r$ thì x_r nó sẽ nằm ở trên một đường thẳng

Bây giờ sử dụng thuật toán local block maching để tìm x_r

Local block maching

Cho hai ảnh stereo đã rectified (trục quang học song song, epipolar nằm ngang), thuật toán Local Block Matching cố gắng tìm dispariy $d$cho mỗi pixel $p = (x, y)$bằng cách:

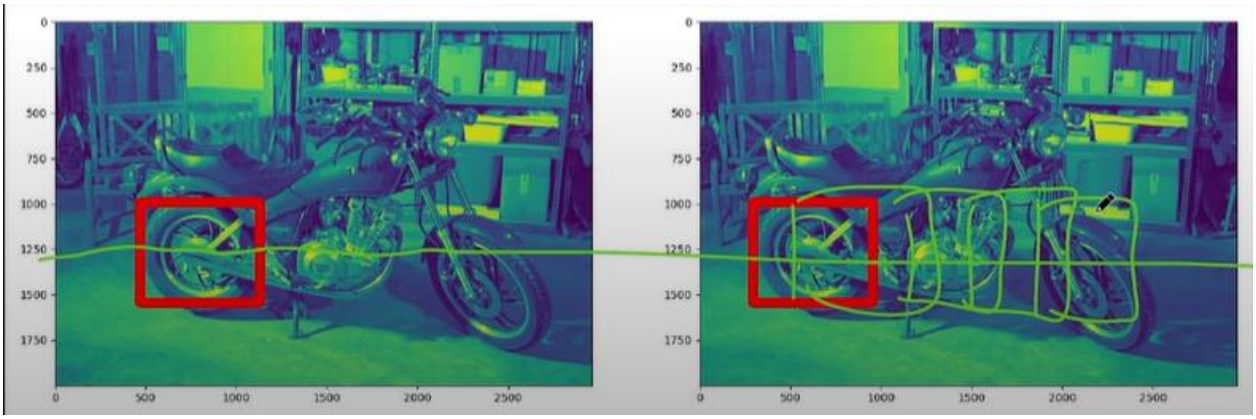
- so khớp từng pixel độc lập,
- dựa trên so sánh độ giống nhau của một cửa sổ nhỏ (block/window) quanh pixel đó,
- và chọn disparity $d$nào có chi phí nhỏ nhất.

Disparity được chọn theo công thức:

$$\boldsymbol{d(x, y) = \arg\min_{d} \; C(x, y, d)}$$

Trong đó $\boldsymbol{C(x, y, d)}$là chi phí khớp (matching cost), Sử dụng hàm so sánh block SAD (Sum of Absolute Differences)

$$C(x, y, d) = \sum_{(i,j) \in W} [I_l(x + i, y + j) - I_r(x + i - d, y + j)]$$



Hình 9. Thuật toán local block matching

Thuật toán khá đơn giản nhưng có nhược điểm lớn là không chính xác nhất và những vùng trơn, ít chi tiết.

Thuật toán SGM Semi-Global Matching

Với thuật toán này nó sẽ sử dụng các thông tin lân cận của pixel ta đoán,

$$S(p, d) = \sum_{r} L_r(p, d)$$

Trong đó, $L_r(p,d)$ là chi phí tích lũy dọc theo đường đi r đến pixel p với độ lệch d.

Hàm chi phí tích lũy: Chi phí tích lũy $L_r(p,d)$ được tính **đệ quy** dọc theo đường đi r,

$$L_r(p,d) = C(p,d) + \min\left(L_r(p-r,d') + P\right)$$

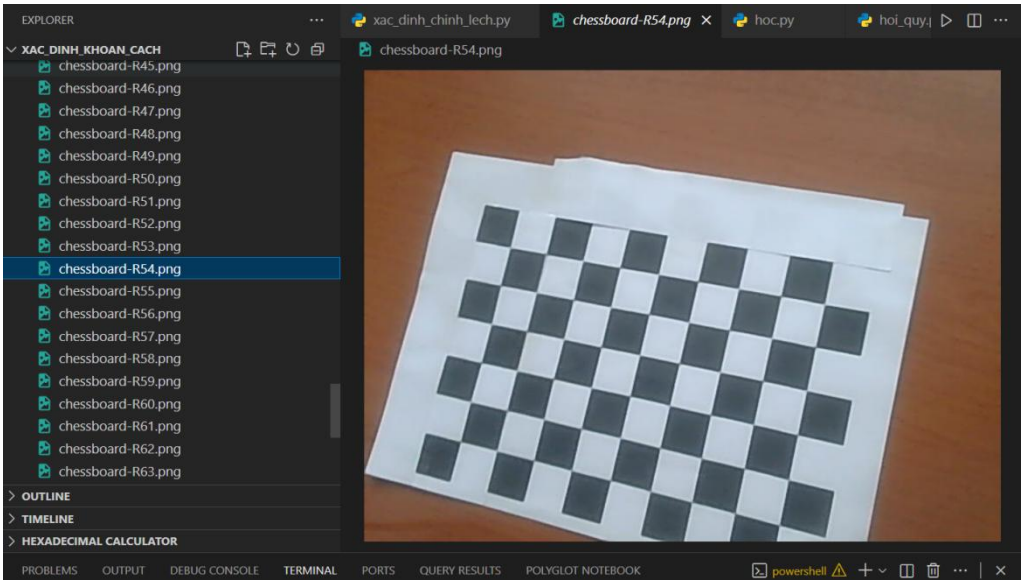
bao gồm:

- **Chi phí khớp ảnh cục bộ** C(p, d).
    - **Hình phạt (P)** cho sự thay đổi độ lệch giữa pixel hiện tại p và pixel liền kề trước đó (p-r) trên đường đi. Có ba mức hình phạt:
    - **P_0 = 0;** nếu d = d'
    - **P_1:** Hình phạt nhỏ cho sự thay đổi độ lệch | d - d'| =< 1 (sự thay đổi giữa hai vị trí trên đường di).
    - **P_2:** Hình phạt lớn cho sự thay đổi độ lệch lớn (| d - d | > 1), khuyến khích bản đồ độ lệch phải mịn (smooth) và chỉ cho phép sự thay đổi độ sâu lớn ở các biên vật thể.
- **Mục đích:** Hình phạt giúp **làm mịn** bản đồ độ lệch và xử lý tốt hơn các vùng **ít chi tiết (low-texture regions)**, vốn là nhược điểm của các thuật toán cục bộ.

Sau đó đi đến ước tính độ lệnh: ta lấy d mà sao cho S(p,d) là nhỏ nhất.

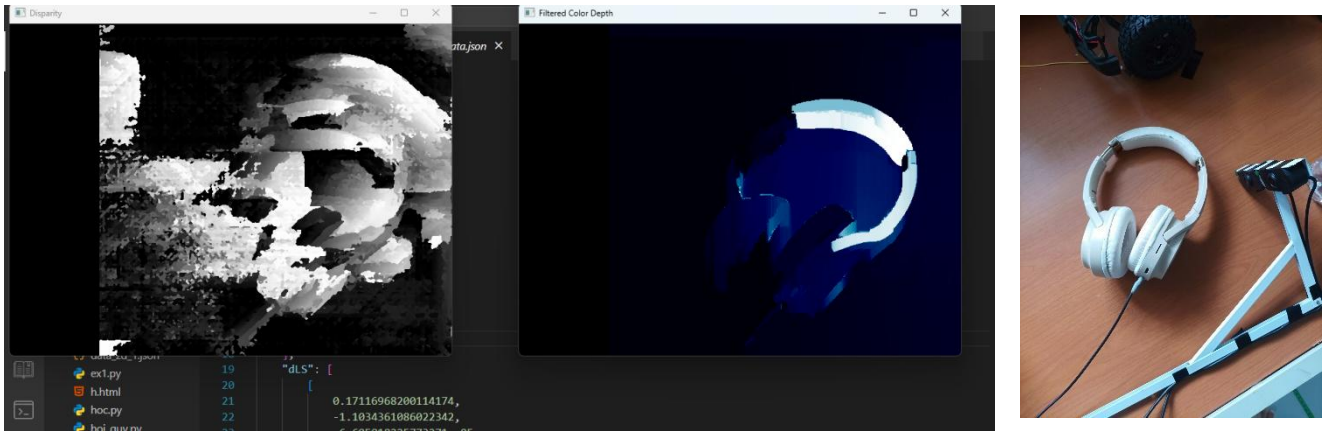
d) Thực hiện.

Hiệu chỉnh camera sử dụng công cụ openCV bằng cách thu thập tất các hình ảnh bàn cờ 10*7 ô với nhiều góc độ khác nhau mục đích là để tính tham số camera chính xác, Xác định chính xác các hệ số méo (lens distortion), Giảm nhiễu và sai số phát hiện góc. Sử dụng cv2.stereoCalibrate để tìm các tham số dựa vào các dữ liệu hình bàn cờ.



Hình 10. Sử dụng bàn cờ hiệu chỉnh camera

Thay vì sử dụng local block maching ở thực tế, dùng semi global block matching (SGBM) vì Local block maching chỉ xét các ảnh hướng xung quanh một điểm nên kết quả không quá chính xác. Kết quả của việc tính ta tìm ra được bản đồ độ chênh lệnh như hình dưới.



Hình 11. Kết quả tạo độ chênh lệch giữa các điểm ảnh

Khi đã tìm được d (độ chênh lệnh) ta sẽ tìm ra Z (độ sâu). $z = \frac{bf_x}{d}$ b (base line) f_x là hai hằng số để tìm được nó không thể xác định bằng cách tìm vào thông số của camera, và đo thủ công khoảng cách giữa hai camera mà ta đặt, Cách tìm là đo mỗi điểm với mỗi khoảng cách, thu được cặp các (z,d) từ đó tìm M

($M = b.f_x$) và tìm giá trị trung bình của nó.

2.6. Giải pháp nhận diện cỏ

2.6.1. Lựa chọn mô hình

Phương thức của đề tài này là đưa vào ảnh và nhận diện cỏ và cây, sau đó xử lí và đưa ra tọa độ để bắn vào cỏ . Như vậy yêu cầu hai vấn đề là nhận diện và đưa ra tọa độ.

Đầu tiên vấn đề nhận diện, vì ảnh là một loại dữ liệu phi cấu trúc. Nó không giống như dạng dữ liệu csv có hàng và cột, dễ dàng xử lí. Mà nó biểu diễn làm một ma trận, với mỗi phần tử là một điểm ảnh (pixel) được sắp xếp theo không gian hai chiều (chiều cao × chiều rộng) và có thể bao gồm nhiều kênh màu (channels). Như vậy rất khó để xử lí với các mô hình xử lí với mô hình Machine Learning thông thường. Nên cần dùng các mô hình Deep Learning với các cấu trúc mạng neural phức tạp giúp có thể học được các đặc trưng từ ảnh.

Vấn đề thứ hai là làm sao để đưa ra tọa độ để bắn cỏ. Để biết được vị trí bắn, thì trước mắt cần xác định được vị trí tổng thể của cây cỏ. Trong các mô hình Machine Learning.

Trong đề tài này chỉ demo việc nhận diện giữa cây rau bạc hà và cỏ dại rồi trả kết quả segmentation (một vùng pixel mà ở đó nhận diện là cỏ) của cỏ, từ đó tính toán để trả về tọa độ điểm bắn theo pixel. Vì dữ liệu không nhiều và rộng nên chỉ tập trung demo một loại là cây rau bạc hà và một số loại cỏ.

Hình 12. Rau bạc hà

Nhận vào hình ảnh và sau đó nhận diện cây và cỏ, trả về bounding box, và segment. Quan trọng ở đây là segment, từ segment tính toán vùng trọng tâm nhất rồi đưa ra điểm trọng tâm cho phần cứng xử lí. Mục đích sử dụng phiên bản segment là vì hệ thống IoT chỉ có một hệ thống chiếu laser để tiêu diệt cỏ, tức là tại một cây cỏ sẽ chiếu 1 hoặc 1 số điểm

mà ở đó thực sự sẽ ảnh hưởng đến cỏ, nên sử dụng phiên bản segment thì sẽ biết được trong box đó chỗ nào thực sự là cỏ. Nếu chỉ sủ dụng box không thì giả sử bắn vào tâm box thì chưa chắc là đã chết cỏ.

Giả sử ảnh đây là kết quả khi chỉ dùng bounding box , và giả sử chỉ có thể bắn một vài điểm cố định cho trước, giả sử tâm box, và sẽ có những trường hợp là không hẳn tâm box là thực sự cỏ.

Hình 13. Kết quả nhận diện cỏ

Và đây là kết quả dùng mô hình segment và từ đó xử lí segment để trả về những vị trí thực sự có thể diệt được được cỏ, những vị trí này tùy thuộc vào vùng segment từ kết quả trả về, không cố định như nếu dùng bounding box.

Hình 14. Tọa độ điểm chiếu laser

Tóm lại đầu ra của bài toán nhận diện trong dự án này là tọa độ các điểm để hệ thống IoT..

2.6.2. Tổng quan về mô hình sử dụng

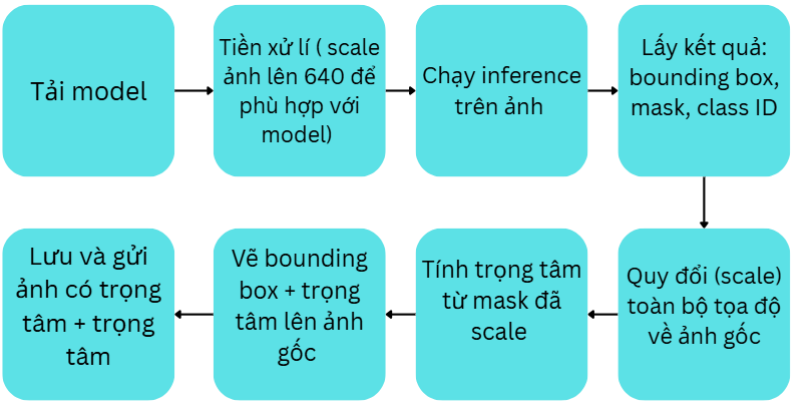
- Vì là cần nhận diện qua hình ảnh, nên mô hình được chọn là một mô hình Computer Vision. Computer Vision (Thị giác Máy tính) không chỉ là một thuật ngữ kỹ thuật; nó là lĩnh vực cho phép máy móc thấy, xử lý, và hiểu thế giới giống như cách con người làm. Từ việc nhận diện khuôn mặt trên điện thoại cho đến việc phân tích hình ảnh y tế, Computer Vision đang biến đổi mọi ngành công nghiệp bằng cách biến dữ liệu hình ảnh (pixels) thành thông tin có ý nghĩa.
- Tuy nhiên, thách thức lớn nhất của lĩnh vực này là tốc độ. Trong thế giới thực, bạn không thể để xe tự lái mất vài giây để nhận ra đèn giao thông. Đây chính là lúc YOLO (You Only Look Once) xuất hiện và thay đổi cuộc chơi.
- YOLO không phải là một mô hình Computer Vision thông thường, nó là tay đua công thức 1 trong thế giới phát hiện đối tượng. Thay vì nhìn một hình ảnh nhiều lần (như các mô hình cũ) để tìm từng vật thể, YOLO xử lý toàn bộ hình ảnh chỉ trong một lần duy nhất. Điều này giúp nó đạt được tốc độ suy luận theo thời gian thực (Real-time Inference) mà vẫn giữ được độ chính xác ấn tượng.
- Với khả năng phát hiện đối tượng (vẽ hộp giới hạn) và phân loại chúng chỉ trong vài mili giây, các phiên bản YOLO (như YOLOv8) đã trở thành công cụ không thể thiếu để tạo ra các ứng dụng đột phá như:
  - Robot Nông nghiệp xác định cây trồng và cỏ dại ngay lập tức.
  - Hệ thống giám sát nhận diện hành vi bất thường trong tích tắc.
  - Trải nghiệm thực tế tăng cường (AR) xử lý môi trường xung quanh bạn không có độ trễ.
- YOLO chính là chìa khóa để mở khóa tiềm năng của Computer Vision trong các ứng dụng yêu cầu tốc độ và sự nhạy bén tối đa!
- Tuy nhiên mô hình YOLO thì có nhiều phiên bản khác nhau, nhưng theo nhiều thí nghiệm và nhiều bài báo thì mô hình YOLOv8 là bản mới và ổn định nhất, ngoài ra còn có YOLOv10,v11 nhưng chúng khá mới và chưa được áp dụng rộng rải, chưa

có nhiều thực nghiệm.

- Như vậy, mô hình hoàn toàn có thể áp dụng thực tế, việc phát hiện nhanh chóng và chính xác, hơn nữa có các phiên bản nhẹ với Jetson Nano và việc có phiên bản segmentation. Đây là một lựa chọn tốt.

2.6.3. Xử lý nhận diện

- Tiền xử lí là một bước quan trọng, mục đích là để phù hợp với khả năng tính toán của chip trên jetson nano, nếu ảnh có kích thước lớn đòi hỏi phải tính toán nhiều hơn dẫn đến quá tải. Và kích thước đưa vào tính toán để phù hợp với cấu hình có thể là giới hạn, nên là cần tiền xử lý để phù hợp với size đầu vào.



Hình 15. Sơ đồ luồng xử lý

- Các tọa độ trọng tâm này sẽ được trả về cho bộ phận IoT xử lí, như vậy tóm gọn nhiệm vụ của việc xử dụng mô hình là xử lí nhận diện, lấy kết quả segment tính toán trọng tâm .
- Việc quan trọng là phải làm sao để model nhận diện được chính xác cỏ và vùng pixel (segment). Phụ thuộc hầu như vào việc train, nói cách khác là dữ liệu có đủ tốt hay không, về mặt số lượng và độ rộng của dữ liệu, chất lượng đầu vào.

2.6.4. Huấn luyện và đánh giá

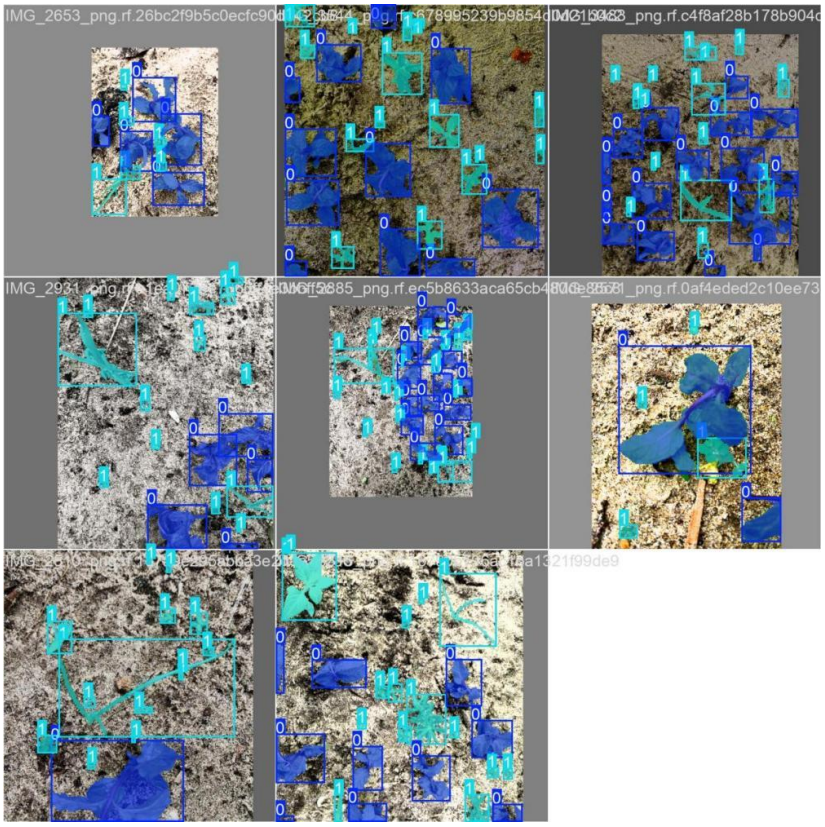
- *Tập dữ liệu 700 : dữ liệu này là ảnh trong đó là một khu vực đất, có cây rau bạc hà và cỏ dại xen kẽ nhau.*

  - Train 75% 525 ảnh

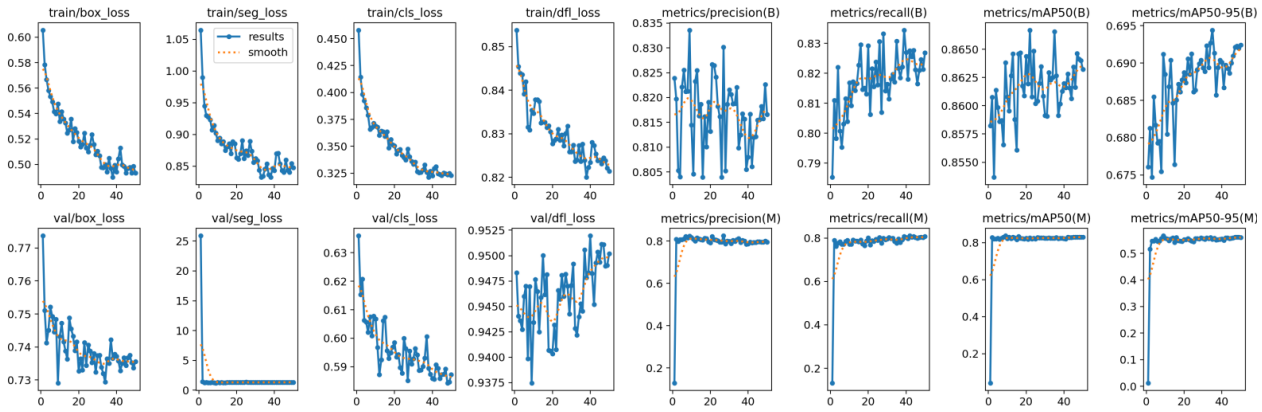
  - Validation (13%) 90 ảnh

- Test (12%) 85 ảnh

- Lô đầu vào batch = 8, con số khuyến nghị khi train với YOLOv8, giữ cho mô hình không học quá nhanh, bỏ lỡ chi tiết.



Hình 16. Lô dữ liệu đầu vào



Hình 17. Kết quả tổng thể huấn luyện

- Có thể thấy box_loss, seg_loss, cls_loss và dfl_loss (độ chính xác của bounding box) giảm đều trên tập train

- Ở tập val thì ở box_loss biến động khá mạnh, nhưng về chung vẫn giảm. Seg_loss thì không giảm nhiều lắm. Cho thấy có thể sai khi dự đoán Segment. cls_loss giảm khá ổn, tín hiệu tốt trong việc đỏ nhầm giữa các đối tượng thành đối tượng khác.
- Precision : Tỷ lệ dự đoán trúng trong tổng số dự đoán là đúng

Ở cả Box và Mask đều cho ra kết quả khá cao là trên 80% .

- Recall: Tỷ lệ dự đoán trúng trong tất cả dự đoán

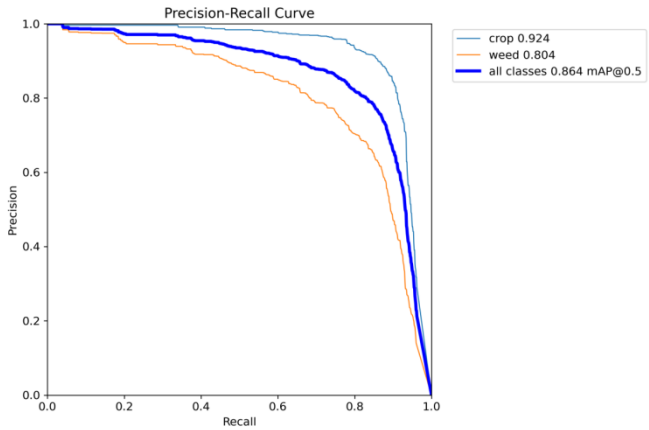
Dù có biến động mạnh ở Mask nhưng cả 2 vẫn cho ra con số là trên 80% khá tốt.

- Về độ chính xác mAP@0.5 thì ở Box khá cáo trên 85%
- Về độ chính xác mAP@0.5 thì ở Mask đạt ngưỡng 80%

☐ Đánh giá trên tập test :

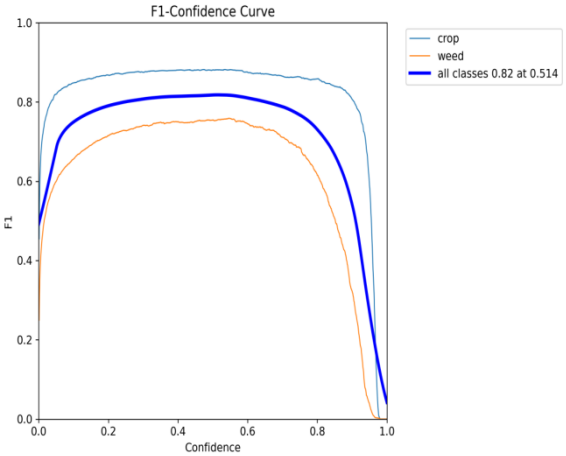
Cho thấy độ chính xác trung bình của các class

Biểu đồ còn thể hiện sự đánh đổi giữa Precision và Recall. Nếu tăng độ tự tín, dẫn đến giảm sai sót thì Precision tăng, nhưng trái lại vì độ tự tin cao nên số đối tượng nhận dạng giảm xuống, điều này làm giảm Recall, nên cần một độ tự tin tối ưu để cân bằng giữa P và R



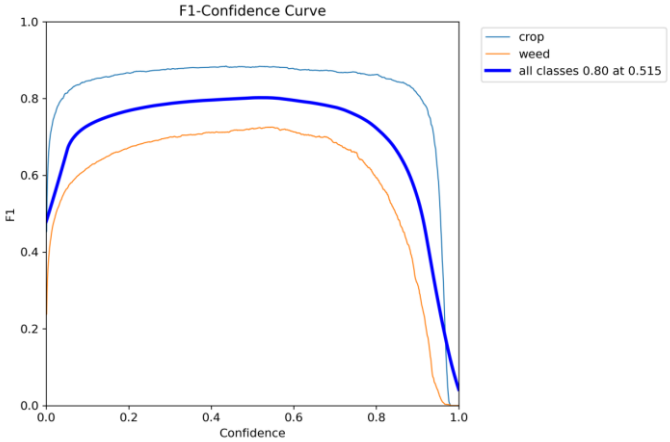
Hình 18. Precision-recall curve

Để biết được Conf bao nhiêu là chính xác thì dựa vào biểu đồ F1-confidence (Box) Biểu đồ này cho biết được ở độ tự tin nào thì Precision và Recall sẽ đạt được độ cân bằng nhất. 0.514 là độ tự tin tốt để đạt được độ cân bằng giữa P và R.



Hình 19. F1-confidence curve

Đó là trên Box, còn trên Mask tương tự ta có được 0.515. Đây là kết quả tốt để vì nó gần sát với Box, tức là có thể dùng chung một độ tự tin.



Hình 20. F1-confidence curve

- Vấn đề quan tâm: xác định số điểm để bắn laser hợp lí

- Việc xác định số điểm để bắn là cực kì quan trọng. Phải bắn làm sao cho vừa tốn ít năng lương và đạt được hiệu quả tiêu diệt .Vì mục đích của mô hình là làm sao tiêu diệt được cỏ nên phải bắn làm sao cho gây ra nhiều tổn hai cho cây nhất. Mức tổn hại cao nhất là cây sẽ chết. Thì đây là hướng tiếp cận rất hợp lí và thực tế.

-Phương pháp1: Cơ bản nhất có thể nghĩ đến là tính toán dựa trên diện tích vùng segment trả về, và cùng với diện tích vòng tròn ảnh hưởng của laser.

+ Cách tính số điểm $Points = S_weed \ / S_laser_circle$ (vòng tròn ảnh hưởng từ laser). Sau đó dùng kmean để phân phối điểm cho hợp lí.

+ Từ số điểm đó, tính ra tỉ lệ diện tích mà laser gây lên cho cho. Bằng cách tính ngược lại, lấy $Points * S_laser_circle/S_weed$ sẽ có được tỉ lệ bao phủ.

+ Nhìn chung cách này luôn đảm bảo cây sẽ luôn chịu sát thương đủ để có thể bị tiêu diệt mà không cần tốn quá nhiều năng lượng.

-Phương pháp 2: Cách này đi sâu vào cấu trúc cây và phân tích xem bắn chỗ nào là hợp lí. Sử dụng thuật toán tìm skeleton (khung xương) Zhang Suen.

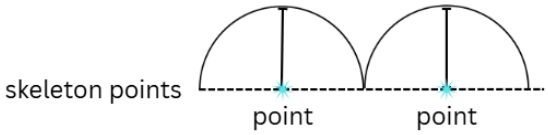
+ Cách hoạt động: Đầu tiên, duyệt qua tất cả pixel, tính khoảng cách của nó đến biên gần nhất, tức là một điểm trên đường chu vi bao quanh. Thì lúc này tất cả pixel sẽ mang giá trị này.

+ Tiếp theo, duyệt qua từng pixel, so sánh giá trị của nó và các giá trị của các pixel lân cận, nếu lớn hơn tất cả thì nó là điểm thuộc khung xương.

P9	P2	P3
P8	P1	P4
P7	P6	P5

Ví dụ: giá trị P1 đang xét lớn hơn các điểm lân cận thì nó thuộc khung xương, và loại bỏ các điểm bên ngoài.

+ Lặp cho đến khi hết pixel, những điểm chưa bị loại bỏ sẽ là khung xương (skeleton) hình dạng của cây cỏ đó. Tuy nhiên kết quả có rất nhiều điểm thuộc không xương và nếu bắn hết thì khá là dư thừa, nên để loại bớt các điểm cho thực tế, cung cấp thêm một biến R (bán kính vòng tròn ảnh hưởng của laser). Tiếp theo chỉ cần duyệt qua các điểm khung xương, và cứ mỗi một đoạn 2*R thì sẽ có một điểm được chọn (để tránh trùng lặp gây lãng phí).

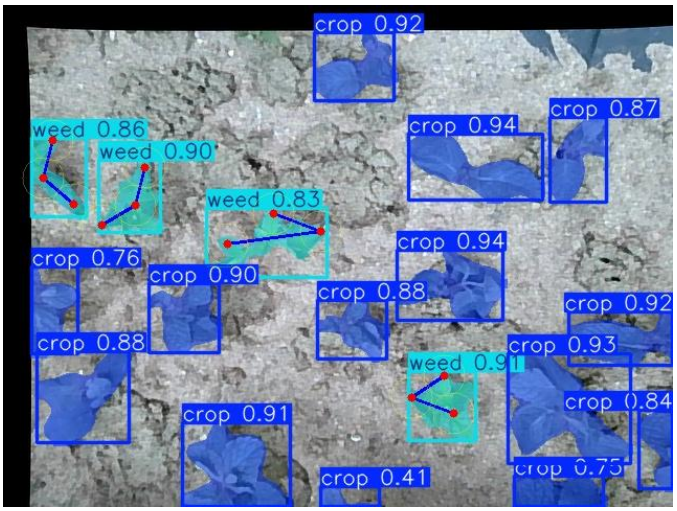


Hình 21. Chia điểm trên khung xương

+ Tương tự từ số điểm đó, tính ra tỉ lệ diện tích mà laser gây lên cho cho. Bằng cách

tính ngược lại, lấy $Points(số\ điểm\ đã\ chia) * S_laser_circle/S_weed$ sẽ có được tỉ lệ bao phủ.

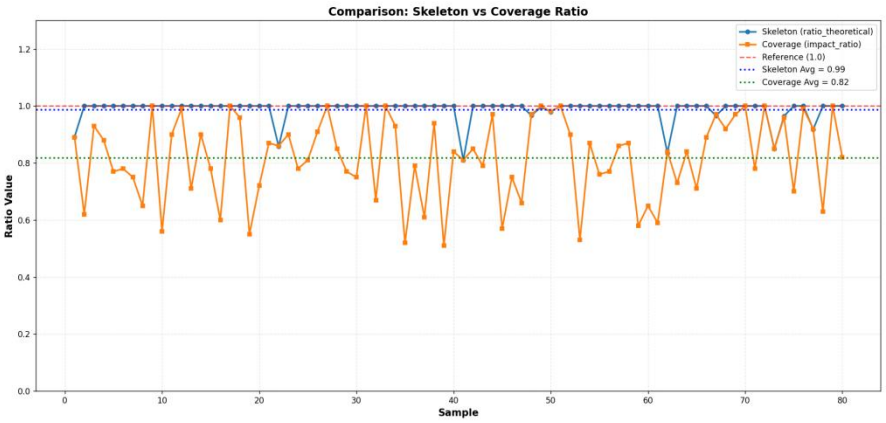
+Kết quả: đã xử lí để hiện lên hình dạng của cây cỏ đó.



Hình 22. Kết quả xử lý

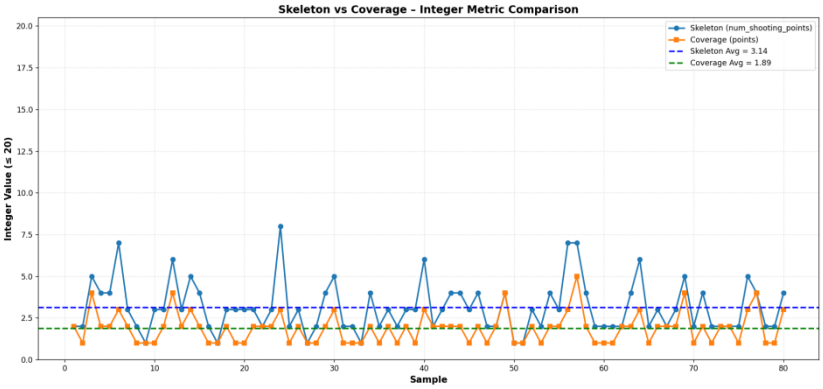
- Đánh giá 2 phương pháp trên:

+ Dưới đây là kết quả theo tỉ lệ cho phủ và theo số điểm bắn của 80 mẫu cỏ nhận diện được. Kết quả che phủ của Skeleton gần như là 100%, và Coverage ở mức trung bình 82% là mức tốt. Nhưng đổi lại số điểm bắn nếu sử dụng phương pháp Skeleton là cao hơn nhiều, điều này nghĩa là tốn nhiều năng lượng hơn.



Hình 23. Kết quả theo tỉ lệ che phủ

- Mỗi phương pháp đề có ưu nhược điểm. Trường hợp thực tế và cỏ lớn thì phương pháp Skeleton có thể tiêu diệt triệt để hơn. Còn trường hợp cỏ nhỏ hoặc demo dự án thì sử dụng cách Coverage thể hiện sự đơn giản và hiệu quả hơn.



Hình 24. Kết quả theo số điểm bắn

Mỗi phương pháp thì có ưu điểm riêng. Trường hợp sử dụng phương pháp skeleton thì có thể tiêu diệt triệt để nhưng khá là dư thừa. Nên trong dự án này đang sử dụng là phương pháp Coverage để có được sự đơn giản, tốc độ và cũng đạt được độ hiệu quả thực tế. Dù không phủ đủ vùng ảnh hưởng của laser nhưng với con số trên 80% ảnh hưởng thì cây cỏ không chết thì ít nhất cũng cực kì chậm phát triển và chết sau đó. Hơn nữa phương pháp này còn giúp tiết kiệm năng lượng.

3. KẾT QUẢ

3.1. Các điều kiện tiến hành thực nghiệm

- **Điều kiện ánh sáng:** Đảm bảo cường độ ánh sáng môi trường ổn định (ưu tiên ánh sáng tự nhiên hoặc có đèn LED trợ sáng), tránh tình trạng ngược sáng hoặc bóng râm quá lớn làm mất chi tiết màu sắc của lá cây.

- **Góc quay:** Camera được bố trí hướng xuống (Top-down view) hoặc nghiêng một góc cố định (khoảng 45-60 độ) so với phương thẳng đứng để thu được hình ảnh toàn diện của tán lá, hạn chế tối đa việc các cây chồng lấn (occlusion) che khuất nhau.

- **Độ ổn định:** Camera được gắn chặt trên khung xe có giảm chấn để giảm thiểu rung lắc khi robot di chuyển, đảm bảo hình ảnh thu được không bị nhòe (Motion Blur) gây ảnh hưởng đến kết quả nhận diện của mô hình AI.

3.2. Thông số kỹ thuật của Jetson Nano khi chạy model

➢ Thông số kĩ thuật của jetson khi chạy với model cần chú ý

- Throughput: 5.04864 qps nghĩa là mỗi giây có thể xử lí được 5 ảnh, điều này hoàn toàn tốt, khi chỉ xử lí một ảnh mỗi lần
- H2D Latency0.57518 ms độ trễ khi gửi dữ liệu từ CPU lên GPU
- D2H Latency0.53381 ms độ trễ khi gửi dữ liệu từ GPU lên CPU
- GPU Compute Time: min = 196.827 ms, max = 199.016 ms, mean = 196.943 ms, median = 196.477 ms, percentile(99%) = 198.689 ms : Thời gian tính toán của GPU với giá trị trung bình đạt mean = 196.943 ms
- Nhìn chung những con sốt này là tốt và chấp nhận được. Trong quá trình test thực tế thì cũng nhận thấy đột trễ khá thấp và thời gian xử lí GPU nhanh, khi mà nếu giả sử đi qua đoạn không có cỏ thì xe hầu như tiếp tục chạy mà không cần dừng lâu, thời gian xảy ra chỉ khoảng 2s.

```
11/21/2025-01:35:13] [I] === Performance summary ===
11/21/2025-01:35:13] [I] Throughput: 5.04864 qps
11/21/2025-01:35:13] [I] Latency: min = 197.139 ms, max = 200.125 ms, mean = 198.056 ms, median = 197.59 ms, percentile(99%) = 199.723 ms
11/21/2025-01:35:13] [I] End-to-End Host Latency: min = 197.15 ms, max = 200.246 ms, mean = 198.072 ms, median = 197.602 ms, percentile(99%) = 199.773 ms
11/21/2025-01:35:13] [I] Enqueue Time: min = 10.8081 ms, max = 23.2109 ms, mean = 17.5384 ms, median = 17.3931 ms, percentile(99%) = 21.9082 ms
11/21/2025-01:35:13] [I] H2D Latency: min = 0.572266 ms, max = 0.578125 ms, mean = 0.57518 ms, median = 0.574219 ms, percentile(99%) = 0.578125 ms
11/21/2025-01:35:13] [I] GPU Compute Time: min = 196.027 ms, max = 199.016 ms, mean = 196.943 ms, median = 196.477 ms, percentile(99%) = 198.609 ms
11/21/2025-01:35:13] [I] D2H Latency: min = 0.535156 ms, max = 0.541016 ms, mean = 0.5381 ms, median = 0.539062 ms, percentile(99%) = 0.540039 ms
11/21/2025-01:35:13] [I] Total Host Walltime: 59.422 s
11/21/2025-01:35:13] [I] Total GPU Compute Time: 59.0829 s
11/21/2025-01:35:13] [I] Explanations of the performance metrics are printed in the verbose logs.
11/21/2025-01:35:13] [I]
&&& PASSED TensorRT.trtexec [TensorRT v8201] # /usr/src/tensorrt/bin/trtexec --onnx=bestV5.onnx --saveEngine=bestV5.engine --fp16 --workspace=2048 --best --
terations=300
-- Quá trình biên dịch hoàn tất! Kiểm tra file best.engine ---
```

Hình 25. Thông số kĩ thuật của jetson khi chạy với model

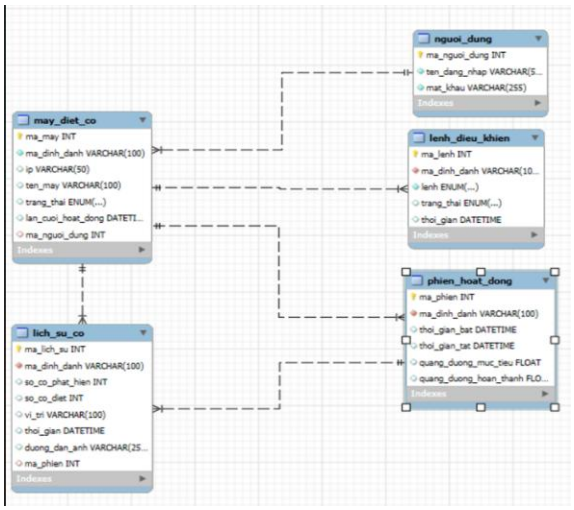
➢ Tóm lại độ chính xác trung bình khi chạy test

- Độ chính xác trung bình với Mask là 0.847 khi độ tự tin là 0.515
- Độ chính xác trung bình với Box là 0.862 khi độ tự tin là 0.514

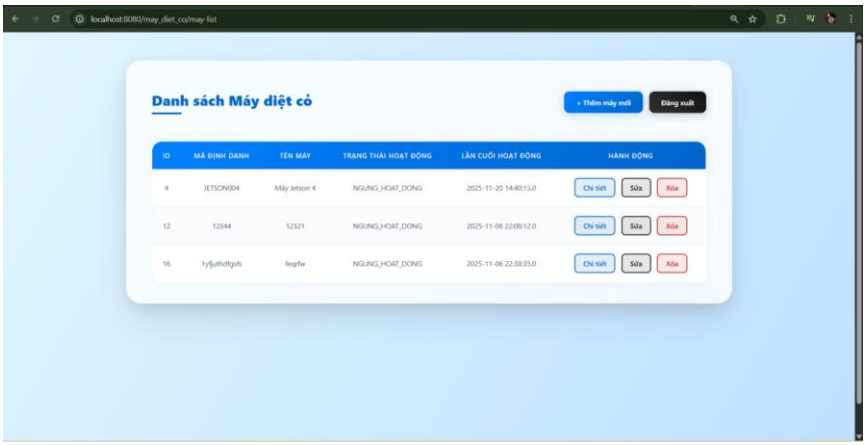
```
Ultralytics 8.3.229 🚀 Python-3.12.12 torch-2.8.0+cu126 CPU (Intel Xeon CPU @ 2.20GHz)
YOLOv8s-seg summary (fused): 85 layers, 11,780,374 parameters, 0 gradients, 39.9 GFLOPs
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100% ━━━━━━━━━━━━ 755.1KB 25.7MB/s 0.0s
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 1409.6±556.6 MB/s, size: 2850.6 KB)
val: Scanning /content/mint-and-weed-14/test/labels... 85 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 85/85 387.6it/s 0.2s
val: New cache created: /content/mint-and-weed-14/test/labels.cache
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95)     Mask(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 6/6 16.4s/it 1
                   all         85       1703      0.849      0.804      0.862      0.691      0.845      0.797      0.847      0.573
                  crop         82        711      0.938      0.869      0.932      0.831      0.942      0.872      0.936      0.742
                  weed         85        992      0.761      0.739      0.792       0.55      0.747      0.721      0.757      0.404
Speed: 5.2ms preprocess, 955.1ms inference, 0.0ms loss, 22.0ms postprocess per image
Results saved to /content/runs/segment/val
💡 Learn more at https://docs.ultralytics.com/modes/val
```

Hình 26. Độ chính xác trung bình khi chạy test

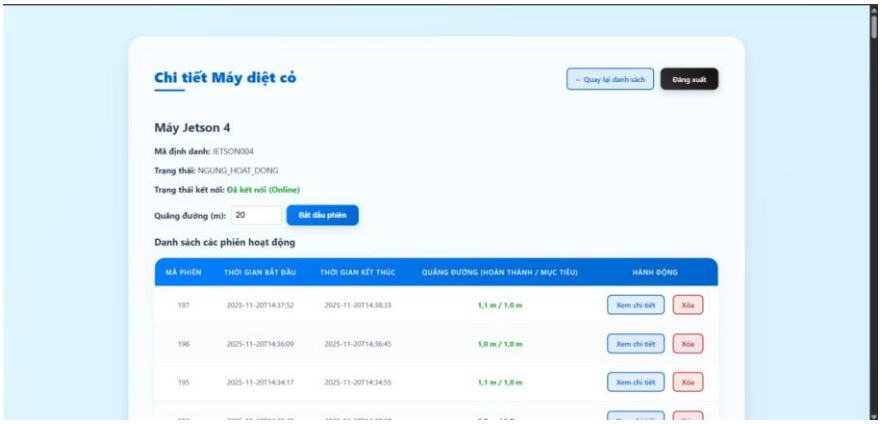
3.3. Kết quả web hệ thống máy diệt cỏ



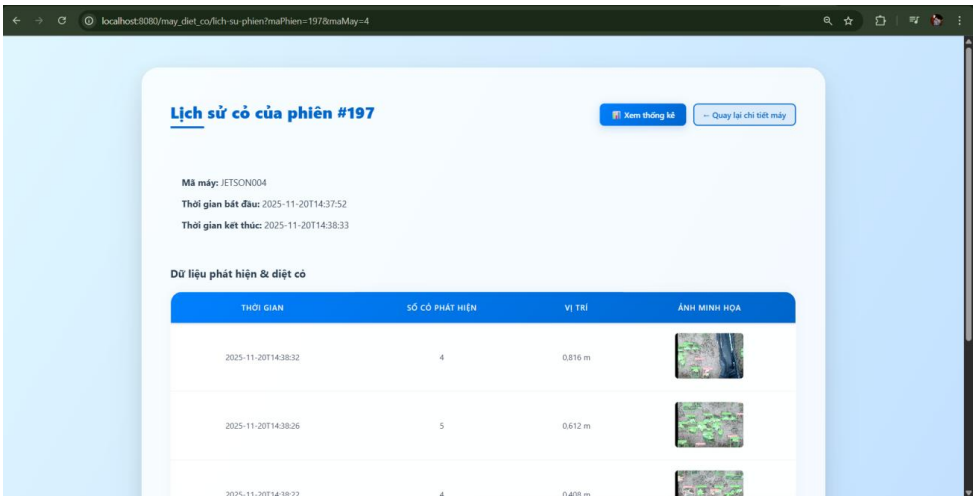
Hình 27. Entity-relationship diagram – ERD



Hình 28. Giao diện trang danh sách máy



Hình 29. Giao diện chi tiết máy khi đã kết nối



Hình 30. Giao diện lịch sử cỏ của phiên

3.4. Đánh giá

3.4.1. Hệ thống IoT

a) Bộ phận đo quãn đường xe.

Cách để biết xe đi bao nhiêm M thì nhóm làm bằng cách dán các mảng nhôm lên bánh xe và sử dụng cảm biến dò line để đếm vạch:

Thông số bánh xe: bán kính 5.4 cm => chu vi bánh xe 34 cm.

sử dụng 5 mảnh nhôm dán vào bánh xe chia thành 5.

Với mỗi vạch là cộng lên 6,8 cm;

Đánh giá độ chính xác cho xe chạy 50cm với 10 lần kết quả là:

45	47	51	55	52	52	53	45	49	46

Giá trị trung bình : 50.5 cm

Độ lệch chuẩn ≈ 3.60 cm

Sai số tuyệt đối trung bình ≈ 3.1 cm

b) Đo khoảng cách bằng camera stereo

đánh giá đo khoảng cách từ camera đên vị trí của cỏ từ hệ thống camera từ stereo.

Mục tiêu: 40 cm (khoảng cách thật giữa camera và cỏ)

Lần đo	1	2	3	4	5	6	7	8	9	10
Giá trị đo	35.8	41.2	39.7	40.9	42.3	39.1	40.4	37.8	41.0	39.4

Sai số tuyệt đối từng lần đo :

Lần	1	2	3	4	5	6	7	8	9	10
Đo (cm)	38.5	41.2	39.7	40.9	42.3	39.1	40.4	37.8	41.0	39.4
Sai số (cm)	1.5	1.2	0.3	0.9	2.3	0.9	0.4	2.2	1.0	0.6

Giá trị trung bình: 40.03 cm

Độ lệch chuẩn : ≈ 1.38 cm

Sai số tuyệt đối trung bình : ≈ 1.13 cm

Sai số lớn nhất: 2.3 cm

Nhận xét: hệ thống stereo camera có thể đo khoảng cách 40 cm với:

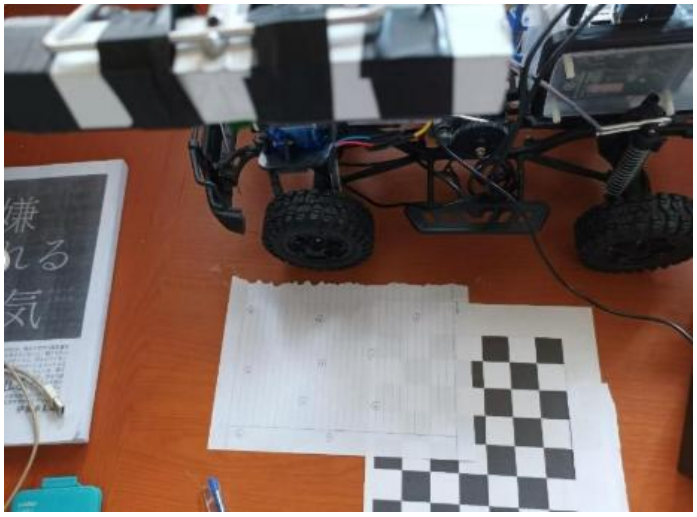
Sai số thấp (~1 cm)

Dao động nhỏ

Độ chính xác trung bình tốt

c) Đánh giá độ chính xác chiếu từ laser đến cỏ.

Với 13 lần đo: kết quả đánh giá là sự chính xác khoảng cách tâm chiếu và laser chiế u đến, bằng cách xác định điểm từ hình ảnh và chiếu laser đến điểm đó.



Hình 31. Đánh giá độ chính xác laser

Dữ liệu thu thập:

Dãy giá trị sai số đo được (cm):

0.6, 0.7, 0.6, 0.4, 0.2, 0.5, 0.4, 0.6, 0.5, 1.0, 1.0, 0.6, 0.7

Tổng số mẫu: 13

Kết quả phân tích thống kê:

$$\text{Sai số trung bình} = \frac{\mathbf{8.8}}{\mathbf{13}} \approx \mathbf{0.6769} \text{ cm}$$

Sai số trung bình ≈ 0.68 cm

Độ lệch chuẩn:

$$\boldsymbol{\sigma} \approx \mathbf{0.23} \text{ cm}$$

Độ lệch chuẩn thấp, thể hiện độ ổn định cao của hệ thống.

Nhận xét và đánh giá:

- Hệ thống cho kết quả ổn định, với sai số trung bình < 1 cm.
- Độ lệch chuẩn chỉ 0.23 cm, thấp hơn mức 0.33 cm của mẫu trước → hệ thống đã cải thiện rõ rệt.
- Sai số phân bố đều, không xuất hiện outlier lớn (ngoại trừ hai giá trị 1.0 cm, nhưng vẫn trong giới hạn chấp nhận).

Nguyên nhân gây sai số:

Một số yếu tố ảnh hưởng trực tiếp:

- Chưa căn chỉnh vuông góc tuyệt đối giữa laser và camera, dẫn đến sai số hệ thống (systematic error).
- Servo MG90S có sai số góc 1–2°, gây lệch hướng chiếu của laser trong một số lần đo.
- Camera stereo có độ chính xác chưa cao, đặc biệt khi tính toán disparity dẫn đến sai số độ sâu.
- Bề mặt vật đo và môi trường ánh sáng có thể gây nhiễu nhẹ đến việc nhận dạng điểm laser.

Kết luận:

- Sai số trung bình của hệ thống hiện tại: ~0.68 cm
- Độ lệch chuẩn: ~0.23 cm
- Kết quả đạt mức độ ổn định tốt, phù hợp cho các ứng dụng đo khoảng cách yêu cầu sai số < 1 cm.
- Hệ thống có thể cải thiện thêm độ chính xác nếu thực hiện:
- Căn chỉnh lại góc chiếu laser cho vuông góc tuyệt đối.
- Tăng độ phân giải stereo hoặc dùng thuật toán lọc nhiễu disparity.

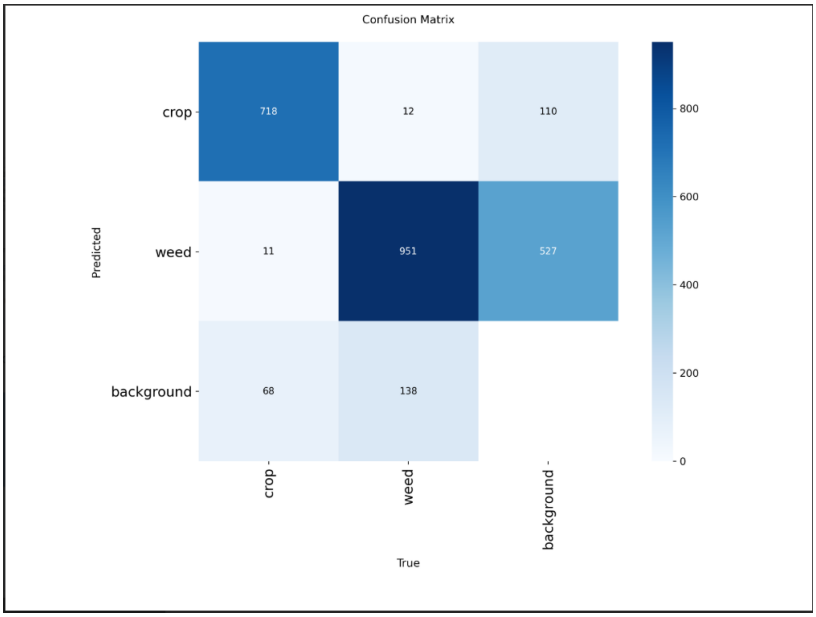
3.4.2. Đánh giá về mô hình AI

```
Ultralytics 8.3.229 🚀 Python-3.12.12 torch-2.8.0+cu126 CPU (Intel Xeon CPU @ 2.20GHz)
YOLOv8s-seg summary (fused): 85 layers, 11,780,374 parameters, 0 gradients, 39.9 GFLOPs
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100% ━━━━━━━━━━━━ 755.1KB 25.7MB/s 0.0s
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 1409.6±556.6 MB/s, size: 2850.6 KB)
val: Scanning /content/mint-and-weed-14/test/labels... 85 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 85/85 387.6it/s 0.2s
val: New cache created: /content/mint-and-weed-14/test/labels.cache
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95)     Mask(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 6/6 16.4s/it 1
                   all         85       1703      0.849      0.804      0.862      0.691      0.845      0.797      0.847      0.573
                  crop         82        711      0.938      0.869      0.932      0.831      0.942      0.872      0.936      0.742
                  weed         85        992      0.761      0.739      0.792       0.55      0.747      0.721      0.757      0.404
Speed: 5.2ms preprocess, 955.1ms inference, 0.0ms loss, 22.0ms postprocess per image
Results saved to /content/runs/segment/val
💡 Learn more at https://docs.ultralytics.com/modes/val
```

Hình 32. Đánh giá mô hình AI

- Tổng thể độ chính xác khi nhận diện cây là ở cả Box và Mask đều trên 0.9 đây là con số được cho là tốt

- Còn cỏ thì con số này còn khá khiêm tốn chỉ ở mức ổn, do đầu vào data set thì cỏ rất đa dạng và đôi khi còn nhỏ nên nhận diện bị nhầm lẫn với môi trường khá nhiều.
- Hình ảnh Confusion Matrix khi test (ma trận nhầm lẫn) : cho thấy con số cỏ bị nhầm lẫn với môi trường là khá lớn.



Hình 33. Confusion matrix khi test

3.4.3. Đánh giá về Web Server

a) Đánh giá về Tốc độ truyền thông (Communication Performance)

Việc áp dụng cơ chế Giao tiếp lai (Hybrid Communication) đã giải quyết triệt để bài toán cân bằng giữa tốc độ và dung lượng:

Kênh Điều khiển (Socket TCP/IP):

- Độ trễ (Latency): Rất thấp, trung bình dao động từ 20ms - 50ms trong mạng nội bộ (LAN).

- Đánh giá: Khi người dùng nhấn nút "Dừng" (Stop) trên Web, Robot phản hồi và ngắt động cơ gần như tức thời. Điều này đảm bảo tính an toàn cao cho hệ thống cơ khí khi gặp sự cố.
- Độ tin cậy: Kết nối Socket được duy trì liên tục (Keep-alive). Cơ chế tự động phát hiện mất kết nối (Heartbeat timeout) hoạt động chính xác, Server lập tức cập nhật trạng thái "Offline" khi Robot bị ngắt nguồn đột ngột.

Kênh Dữ liệu (HTTP REST API):

- Tốc độ tải lên: Với hình ảnh độ phân giải HD (720p, ~100KB/ảnh), thời gian upload trung bình khoảng 300ms - 800ms tùy thuộc vào chất lượng sóng Wi-Fi.
- Đánh giá: Tốc độ này hoàn toàn đáp ứng được yêu cầu lưu trữ bằng chứng. Việc tách luồng upload ảnh sang HTTP giúp luồng Socket điều khiển không bao giờ bị tắc nghẽn (Block) bởi dữ liệu lớn.

b) Đánh giá về Máy chủ Web (Backend Stability)

- Khả năng chịu tải: Máy chủ Apache Tomcat kết hợp với kiến trúc Đa luồng (Multi-threading) của Java cho phép xử lý đồng thời nhiều tác vụ: vừa nhận lệnh từ Web, vừa lắng nghe Socket từ Robot, vừa ghi dữ liệu vào MySQL mà không xảy ra hiện tượng "treo" hoặc xung đột tài nguyên.
- Quản lý giao dịch (Transaction): Các thao tác với Cơ sở dữ liệu (đặc biệt là chức năng Xóa phiên và Lịch sử) hoạt động chính xác, đảm bảo tính toàn vẹn dữ liệu, không để lại dữ liệu rác.

3.5. Bảng thống kê chi phí

Linh kiện	Chi phí
Jetson Nano	5.600.000 (Mượn khoa)
Xe địa hình	1.000.000 (Mượn khoa)

Webcam x2	340.000
Arduino	70.000
Servo x2	90.000
Laser	10.000
Cảm biến dò line	20.000
Module Relay x2	48.000
Pin cấp nguồn	220.000
Tổng	7.398.000

4. HƯỚNG PHÁT TRIỂN

4.1. IoT

- Sử dụng lấy ảnh thông qua camera conn đển giải quyết vấn đề độ trể đọc camera.
- Trên thực tế nên dùng nhiều laser phân bổ ra nhằm mục đích chiếu chính xác hơn, tránh các điểm khuất bị che bởi trồng.
- Hiên tại xe mới demo là đi thẳng chưa nhận diện luốn đi, chính vì thế cần phải thêm hệ thống nhận diện luống ruộng để xe di chuyển chính xác.
- Tăng tốc độ diệt cỏ bằng cách xe đi liên tục và xử lý tại từng điểm phát hiện trên camera.
- Phát triển khung xe cho đúng với thực tế, vì hiện tại chỉ mới dừng ở thử nghiệm.
- Hiện tại laser chỉ nhằm mục đích chỉ đúng vị trí ta cần diện vậy nên hướng phát triển tăng công suất laser để có thể đốt cháy được cỏ.

4.2. Web Server

- **Nâng cấp kiến trúc Real-time Web (WebSocket):**
  Cập nhật trạng thái và vị trí Robot theo thời gian thực, hiển thị trực tiếp trên Dashboard và gửi cảnh báo tức thì khi có sự cố.
- **Mở rộng truy cập Internet và điện toán đám mây:**
  Public hệ thống ra Internet và triển khai trên nền tảng đám mây, cho phép giám sát và điều khiển Robot từ xa thông qua mạng 4G/5G.
- **Tích hợp bản đồ số và phân tích dữ liệu nâng cao:**

Hiển thị vị trí Robot trên bản đồ ruộng và xây dựng Heatmap xác định khu vực nhiễm cỏ để hỗ trợ ra quyết định canh tác.

- **Chuyển đổi sang Progressive Web App (PWA):**
  Cho phép cài đặt và sử dụng hệ thống như ứng dụng di động, hỗ trợ cache dữ liệu và hoạt động offline cơ bản mà không cần phát triển ứng dụng riêng.

4.3. Model AI

- Thu thập nhiều data set hơn để hướng mô hình đến việc có thể nhận diện được trong nhiều trường hợp cây cỏ chứ không phải chỉ cây rau bạc hà.
- Tối ưu độ chính xác bằng cách thêm các thông số độ nhiễu và thay đổi khi train nhằm cho mô hình không học thuộc lòng quá mức dẫn đến kết quả khi test bị sai , hơn nữa chất lượng data set phải giống với chất lượng từ cam chụp, thì xử lí sẽ tăng độ chính xá

TÀI LIỆU THAM KHẢO

[1] H. Hirschmuller, "Stereo Processing by Semi-Global Matching," *IEEE Transactions on Pattern Analysis and Machine Intelligence,* pp. 328-341, 2008.

[2] S. Vujasinovic, "Stereo-Vision (source code)," 2018. [Online]. Available: https://github.com/LearnTechWithUs/Stereo-Vision.

[3] S. Sonawane and N. N. Patil, "Crop-Weed Segmentation and Classification Using YOLOv8 Approach for Smart Farming," *Journal of Studies in Science and Engineering,* 2024.

[4] A. S. Foundation, "Apache Tomcat – Version 10.0 Documentation," 2025. [Online]. Available: https://tomcat.apache.org/tomcat-10.0-doc.

[5] Oracle, "Java™ Tutorials – Sockets," 2025. [Online]. Available: https://docs.oracle.com/javase/tutorial/networking/sockets.

[6] NVIDIA, "Jetson JetPack Documentation," 2025. [Online]. Available: https://docs.nvidia.com/jetson/jetpack.